

15-march-2026

NodeJS Internals and Architecture

How NodeJS works

- Browser have engine to run JavaScript

Browser engine example,

chrome = V8

Firefox = SpiderMonkey

Apple (Safari) = JavaScriptCore

- JavaScript is a simple languages,
it only has console.log or you create function
or run.

- JavaScript doesn't have any advanced features
ex.

you can't call API in JavaScript.

JavaScript doesn't have setTimeout or setInterval

JavaScript doesn't have asynchronous

there is no web API in JavaScript

- JavaScript core language is simple,

It's do,

you can create a function and call them

or create a variables and define them

or add the two numbers

- The functionality we add on JavaScript like
DOM, setInterval, fetch, async Promises those are
implement by browser.

Note! - even console.log not a part of JavaScript.

Date _____
Page _____

- JavaScript need a base to run outside the browser.
so, the base is C++

JS Engine

C++

JavaScript is embedded by ~~C++~~ in C++, then we can run C++ in over local machine as a runtime, so that's ~~the~~ why we can also run JS :

- Ryan Dahl who is a developer of NodeJS
- But it's just a plain JS when it existed the browser
- that's why he imported libuv and libuv adds Event loop & Threadpool

So, when V8 engine from google chrome and C++ and libuv are combined then it's called NodeJS

$$V8 + (C++) + Libuv = NodeJS$$

- The way JavaScript run in browser it's different and
- The way JavaScript run in nodeJS environment is different
if output is same but it's run internally differently

Note 1 Standard output means output your

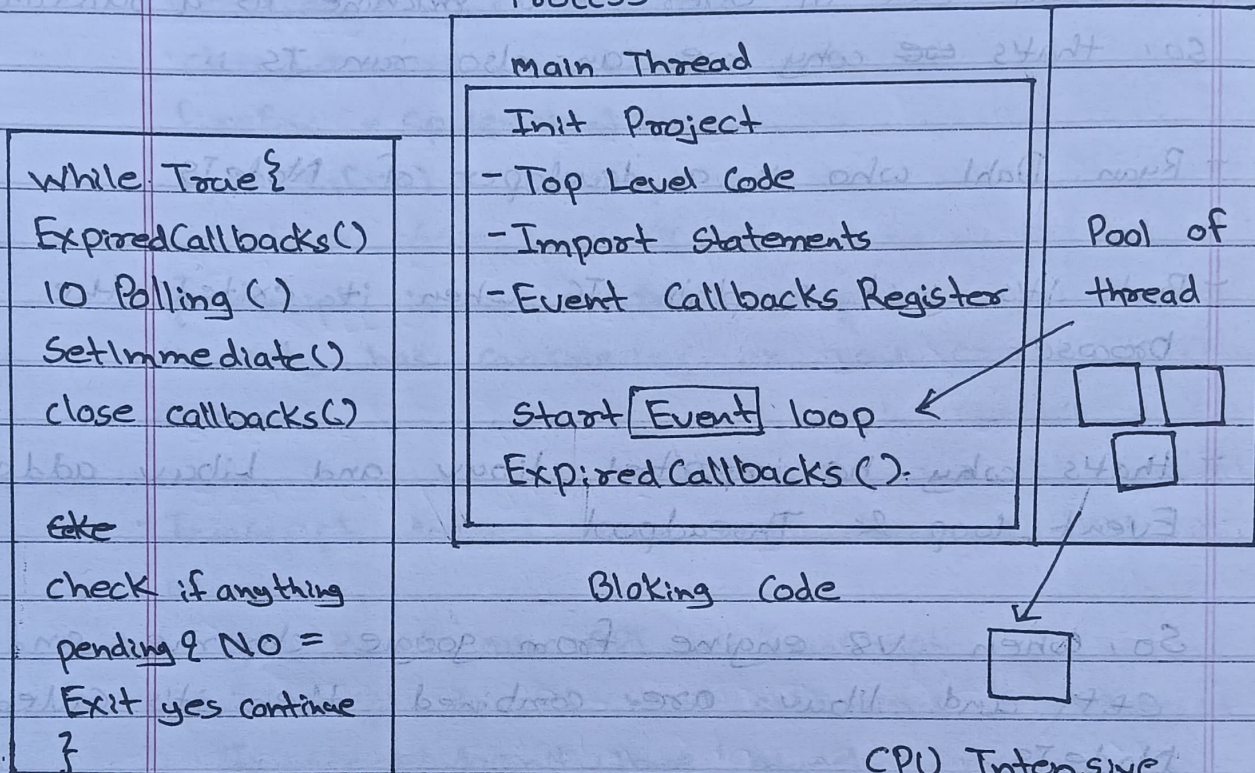
terminal, jese browser alog kis window me jake render karta hai - Kyuki console.log javascript ka part hi nahi hai, it's a part of browser.
syntax is same but internally are different

Note- The way JavaScript run in node environment not same as browser.

How NodeJS is run

When you run node filename.js
(tab ek node name ki Process banti hai)

Process



CPU Intensive Task

Cryptography

Reading

Encryption

- main thread where all JavaScript code executes.

(If this thread got killed or block then your code stops running)

- main thread is important part of your code.

- This is the only thread we have, if you killed it or blocked it then your code is not going to work.

There is steps to run code in NodeJS.

- 1) first initialise the project

Init Project

- 2) then first of all Top level code runs

(top level code = koi bhi yesa code jo kisi function ke under nahi hai ke under nahi hai, or yaha direct likha huva hai.)

ex,

```
console.log('Hello from NodeJS');
```

```
const a = 2+2;
```

```
console.log('a', a);
```

(basically sabse pehle top level code run karta hai)

(If, else, loops bhi ~~challenge~~ Import also)

- 3) after that your Event Callbacks are registers

- 3) after that all your Import statements run (Import)

- 4) then your all Event Callbacks registers

ex,

jese aapko kisi 'SIGINT' ka event milta hai.

```
process.on('SIGINT') => {
```

```
  console.log('SIGINT');
```

```
}
```

Note: When you run server on express, vo server hamesh chalta rehta hai, use hum `ctrl + c` se exit karte hai, isse vo process band ho jati, matlab wo ek signal bhejti hai, jise.

vo signal process ko gracefully shut down
kar dete hai

Some keys of signals

- SIGNAL
- SIGHUP (signal 1): Hangup
- SIGINT (signal 2): Interrupt (usually sets by `Ctrl+C`)
- SIGQUIT (signal 3): Quit
- SIGKILL (signal 4): kill (cannot be caught or ignored)
- SIGTERM (signal 5): Termination Signal
- SIGSTOP (signal 6): stop the process (cannot be caught or ignored)

ex,

```
process.on('SIGINT', () => {
```

```
  console.log('do the cleanup...');  
  process.exit();  
});
```

clean up ke console.log ('do the cleanup...');
pehle pura code process.exit();
exit hoga });

(ye aapke sab event call ko memory me
register kar deti hai

register - store karنا / backup the mind karنا

5) then it's start's the Event loop

- iske bad loop chalu ho jata hai
- ab kabhi bhi aapka top level code,

Important statement: Event callback register

nahi ho sakte

Event loops

Kyuki vo ab unreachable ho gaye hai loop start hone ke bad.

1) Sabse pehle koi expired-callbacks unko run kardo

```
While True {
```

```
  ExpiredCallbacks ()
```

```
  }
```

ex,

```
import fs from "fs"; Import statement
```

```
console.log("Hello from NodeJS"); top level code
```

```
setTimeout(() => console.log("Hello from setTimeout"), 0);
```

↓ timer
↓ callback function

```
const a = 2 + 2; top level code
```

```
console.log('a', a); top level code
```

output

Hello from NodeJS

a 4

Hello from setTimeout

Note:- Sabse pehle top level code chahenge.
uske bad Import statements.

callback ready to run hai, per vo nodeJS ka part nahi isliye vo event loop start hone ke bad hi chalega. (Timer expire - pehle hi ho jata hai)

Note:- Event loop sabse pehle timers check karti hai.

Interviews question

timer kitne time bad chalegi

ex,

```
import fs from 'fs';
```

```
console.log('Hello from NodeJS');
```

```
setTimeout(() => console.log('Hello from setTimeout'),
```

// Timer start karna

```
2 * 1000, ); // callback function
```

2 sec

```
const a = 2 + 2;
```

```
console.log('a', a);
```

```
console.log('a', a);
```

```
console.log('a', a);
```

```
console.log('a', a);
```

```
console.log('a', a);
```

// 10 million log (50 seconds)

Output

Hello from NodeJS

a a

a a

a a

a a

a a

a a

a a

a a

a a

question:- agar top level code chahiye me 50 sec lagne to timer kitne time bad chalegi

answer:- timer chalega top level code run hone ke bad kyun ki, timer 2sec ka tha or vo background me chal rha tha, agar vo ex 60 sec ka hota to top level code 50sec ka run ho jane ke bad 10 sec bad run hata

2) IO Polling (Input output task)

Agar apne koi file reading pe lagaya tha uspe bhi koi callback register hoga, jab file reading complete ho jaye tab callback run karna hai)

ex.

Import fs from 'fs';

console.log('Hello from NodeJS');

→ top level code hai

fs.readFile('sample.txt', 'utf-8', (err, data) => {
 console.log('file reading complete');
});

↓
ye callbacks hai

setTimeout (

() => console.log('Hello from setTimeout'), 0);

const a = 2+2;

console.log('a', a);

Output

Hello from NodeJS

a 4

Hello from setTimeout

file reading complete

even though setTimeout niche tha par pehle fs tha, tabhi bhi fs bar me aaya kyun ki

→ kyun ki event loop ka design hai, pehle top level code, uske bad event's loop chelegi, tab pehle callbacks fir uske bad IO polling

3) setImmediate()

- This function only available in NodeJS,
Browser me nahi rehta hai.

ex,

```
import fs from 'fs';
console.log("Hello from NodeJS");

fs.readFile("sample.txt", "utf-8", (err, data) => {
  console.log("File reading complete");
});

setImmediate(() => console.log("Hello"));
```

```
setTimeout(() => console.log("Hello from  
setTimeout"), 0);

const a = a + a;
console.log('a', a)
```

Output

```
Hello from NodeJS
a4
Hello from setTimeout
file reading complete
Hello
```

Note:- agar file reading me time lag to bad me bhi run ho sakta hai.

diagram :- while True {

Expired Callbacks()

IO Polling

setImmediate()

close callbacks

}

4) close callbacks

- aapne jese koi socket laga rakha hai, agar vo disconnect hoga to use run kardo, agar koi file reading close ho rahi ho to use run kardo, server ke close callbacks vo run hote hai basically.

Note - par ye 4 steps to while loops me aate hai, or koi base condition to hai hi nahi, ye to hamerha fir loop me phans jayega, process khatam hi nahi hoga tab to,

Isliye ek base condition lagani jati hai, jese

While True {

ExpiredCallbacks()

IO Polling()

setImmediate()

close callbacks()

check if anything

pending? No = Exit

Yes continue

- agar koi expired callbacks hai, ya IO polling task chal raha hai background me, ya setImmediate ka function hai to continue karo.
- agar kuch pending nahi hai to exit kar jao.

ex agar aap ne setTimeout 30sec ka de diya,

tab event loop bar bar check karte rahi gi tab tak 30 sec pare hoke setTimeout nahi run ho jati tab tak.

Agar event loop ke under ek blocking code likh du to,

ex. jese IO polling huva koi file reading operation.

```
fs.readFile('sample.txt', 'utf-8', (err, data) => {
  console.log('File reading complete');
  while (true) {}
});
```

iske karan event loop forever IO polling pe hi atk jayegi. isse hi blocking code kehte hai. or iske bad koi bhi code chul ni sakte ab

Note: Don't write blocking code on server.

Treadpool

- agar file heavy huvi, to kab run hogi. Kyu ki JavaScript to single thread language hai. Sub ek single process pe hi chalega matlab
- Isliye libuv me ek thread pool bhi diya hai. event loop ke saath
- to JavaScript ke pass bhi multiple thread hote hai with the help of libuv
- matlab ek workers ka pool hota hai. ise pool of thread kehte hai, yaha sare thread free hote hai
- By default **4** hote hai

Par NodeJS ka design kya kehta hai.

- agar koi CPU Intensive Task aa gaya

- **Cryptography**

Reading

Encryption

Cryptography task

- In backend development, cryptography is the practice of securing data through mathematical algorithms.

It ensures that sensitive information remains confidential, integral, and authentic as it moves between a user's browser and your server, or while it sits in your database.

- To aag jab koi CPU Intensive task aaata hai, event loop khud na karke thread ko de deta hai, tab thread pool kam pe lag jati hai

- par event loop thread pool ko bar bar puchhti rehti hai, ki tumhara kam huva, huva, isliye ki ise polling kehte hai bar bar puchhne ko hi.

- By default 4 thread hote hai, ap use increase bhi kar sakte ho.

- But you can't decide which task is given event loop or the thread pool.

It's already decide by nodeJS.

- agar cpu task to thread pool
- agar synchronous task to event pool run karegi

DRY Run Code

examples 1

```
import fs from 'fs';
```

```
setTimeout ( ← expired callbacks
```

```
( ) => console.log ('Hello from Timer', 0);
```

```
console.log ('Hello from Top level code'); // top level code
```

output

Hello from Top level code

Hello from Timer

example 2

```
import fs from 'fs';
```

```
setTimeout (
```

```
( ) => console.log ('Hello from Timer', 0);
```

```
setImmediate ( ( ) => → setImmediate function
```

```
console.log ('Hello from Immediate', 0);
```

```
console.log ('Hello from Top Level code');
```

output

Hello from Top Level code

Hello from Timer

Hello from Immediate

example 3

```
import fs from 'fs';
setTimeout(() => console.log('Hello from Timer'), 0);
setImmediate(() => console.log('Hello from Immediate'), 0);

fs.readFile('sample.txt', 'utf-8', function(err, data) {
  console.log('File Reading Complete...');
}); // IO polling
```

console.log('Hello from Top level code');

output

```
Hello from Top level code
Hello from Timer
Hello from Immediate
File reading complete.
```

Note:- you can't predict IO polling time because you don't know how much hang is file. that's why setImmediate run before IO polling.

example 4

```
import fs from 'fs';
setTimeout(() => console.log('Hello from Timer'), 0);
setImmediate(() => console.log('Hello from Immediate'), 0);
```

```
fs.readFile('sample.txt', 'utf-8', function (err, data) {
  console.log('File Reading Complete...');

```

```
  setTimeout(() => console.log('Time 2'), 0);

```

```
  setTimeout(() => console.log('Time 3'), 0);

```

```
  setImmediate(() => console.log('Immediate 2'), 0);
});
```

```
console.log('Hello from top level code');
```

Output

Hello from top level code

Hello from timer

Hello from Immediate

File reading complete

Immediate 2

Time 2

Time 3

Note:- Sabse pehle top level code chalega uske

baad event loop chalega fir uske baad

usme sabse pehle expired callback.

So phir second phase me IO polling, fir

IO polling me file read hone me time lagega

fir phase 3 setImmediate run ho jayega

fir uske baad event loop fir check karega,

jab file reading complete hogi tab vo chal

padega, par ab hab IO polling ke phase 2

me hai, to fir phase 3 setImmediate ko

dekhega fir vo chal padega, fir uske baad loop

chalega baki ka bacha expired callbacks chalega

or loop fir expired ho jayega.

example 5

```
import fs from 'fs';
```

```
import crypto from 'crypto';
```

```
const start = Date.now();
```

```
setTimeout(() => console.log('Hello from Timer'), 0)
```

```
setImmediate(() => console.log('Hello from Immediate'), 0)
```

```
fs.readFile('Sample.txt', 'utf-8', function(err, data) {  
  console.log('File Reading Complete...');
```

```
setTimeout(() => console.log('Time 2'), 0);
```

```
setTimeout(() => console.log('Time 3'), 0);
```

```
setImmediate(() => console.log('Immediate 2'), 0);
```

```
crypto.pbkdf2('password', 'salt', 300000, 1024, 'sha256',  
(err) => { console.log('Password 1 has been hashed',  
  Date.now() - start);
```

// Password hashing

```
crypto.pbkdf2('password', 'salt', 300000, 1024, 'sha256',  
(err) => { console.log('password 2 has been hashed',  
  Date.now() - start);
```

```
crypto.pbkdf2('password', 'salt', 300000, 1024, 'sha256',  
(err) => { console.log('password 3 has been hashed',  
  Date.now() - start);
```

```
crypto.pbkdf2('password', 'salt', 300000, 1024, 'sha256',  
(err) => { console.log('password 4 has been hashed',  
  Date.now() - start);
```

Note- Hum password ko 4 bar hash kar bahe, he isliye
4 workers kam pe lag jayenge.

Date _____
Page _____

`console.log('Hello from top level code');`
output

Hello from top level code

Hello from timer

Hello from Immediate

File Reading Complete

Immediate 2

Time 2

Time 3

Password 3 has been hashed 902

Password 2 has been hashed 1240

Password 4 has been hashed 1304

Password 1 has been hashed 1403

4 ko kam

almost sat nehi

Khataam hua.

agar aap ne ek or password add kijiye to
use chal ne me 1st se double time lagega,
keru ki jab vo free hoga tabhi hi 5th no
ke password ki hashing dekhega na, isliye.

Thread ki size ko hum update bhi kar
sakte hai.

~~Example~~

`Process.env.UV_THREADPOOL_SIZE = 3;`

- This is the how you increase and decrease the
size of thread pool workers.
and it's depend on How many core you have in
your system

Example 6

```
import fs from 'fs';
```

```
setTimeout(() => console.log('Hello from Timer'), 0);
```

```
setImmediate(() => console.log('Hello from
```

```
Immediate'), 0);
```

```
console.log('Hello from top level code')
```

output

Hello from top level code

Hello from Timer

Hello from Immediate.

example 7

```
import fs from 'fs';
```

```
setTimeout(() => console.log('Hello from Timer'), 0);
```

```
setImmediate(() => console.log('Hello from
```

```
Immediate'), 0);
```

output

Hello from Immediate

Hello from Timer.

Why both the result is different because of

console.

behavior of console.log present and when it's absent.

Because,

The behavior of `setTimeout` and `setImmediate` depends on where they are called.

- `setTimeout(Fn, 0)`

Technically has a minimum delay (usually 1ms).
It belongs to the Timers phase.

- `setImmediate(Fn)`

Designed to execute immediately after the poll phase completes.

The `console.log` effect

When you add `console.log` or other synchronous code, you are effectively slowing down the entry into the event loop.

- without `console.log`

The script initializes and enters the event loop almost instantly. The timer is in a race against the clock (literally).

- with `console.log`

Synchronous operations take time. Even though `console.log` is fast, it takes a few fractions of a millisecond to initialize the output stream and write to the terminal.